

Alpha-GPT 3.0 — Engineering Reference

From raw market data to a backtested portfolio of machine-generated alphas

Internal engineering notes

Contents

1	What the system is	1
2	Repository map by stage	2
3	Stage 0 — Dataset construction	2
3.1	Sources	2
3.2	Point-in-time, survivorship-free universe	2
3.3	Derived panels	3
3.4	Terminal discovery + IC cap	3
3.5	Anti-look-ahead controls (summary)	3
4	Stage 1 — Idea generation	3
5	Stage 2 — Formula generation & the alpha language	4
6	Stage 3 — Verification	4
7	Stage 4 — Evaluation (train/val) & metrics	4
8	Stage 5 — GP amplification (optional)	5
9	Stage 6 — Storage	5
10	Stage 7 — Repeat N times (loop & parallelism)	5
11	Stage 8 — Portfolio construction	5
12	Stage 9 — Portfolio backtest & metrics	6
13	End-to-end runbook	6
14	Empirical result (first unpolished 1,000-alpha run)	6
15	Reading the results — is Sharpe 0.88–1.4 “good”?	7
16	Limitations & roadmap	7

1 What the system is

The system has two macro-phases.

1. **Dataset build** (offline, run occasionally). Turn raw CRSP/Compustat/OSAP into a clean, point-in-time (PIT), survivorship-free panel library — one `date` $\times$ `PERMNO` float matrix per data field. These fields are the *terminals* of the alpha language.
2. **Alpha factory** (run repeatedly). Autonomously generate alpha *expressions*, verify them, evaluate them on train/validation, store them in a database; then filter, combine many of them into a single portfolio signal, and backtest that portfolio on a held-out test period.

```
DATASET BUILD
CRSP CIZ (26GB), Compustat (2.3GB), OSAP signals
--> PIT survivorship-free universe (annual rebalance)
--> price/vol panels + cs.* fundamentals + osap.* signals + sector aux
--> coverage report + terminal discovery / IC-cap
==> data/panels/*.parquet, data/universe/membership.parquet

ALPHA FACTORY (loop N)
idea (LLM) --> formula(s) --> VERIFY --> EVALUATE(train+val) --> STORE(SQLite,dedup)
optional: multi-agent Idea+Formula Debate; optional: GP amplification

PORTFOLIO (offline)
filter survivors (val metrics) --> re-eval on TEST --> sign-flip+zscore
--> combine (1/N | IC-weighted | decorrelated | orthogonalized) --> backtest on TEST
```

Golden rule (leakage). Selection happens on **train/validation**; the **test** split (2021–2023) is only ever touched by the final portfolio backtest.

2 Repository map by stage

Stage	Files
Dataset: universe	scripts/prepare_data.py, alpha_gpt/data/universe.py
Dataset: fundamentals	scripts/ingest_fundamentals.py, alpha_gpt/data/fundamentals.py
Dataset: OSAP	scripts/ingest_osap.py, alpha_gpt/data/osap.py
Dataset: quality/aux	alpha_gpt/data/coverage.py, analysis/neutralize.py, scripts/verify_data.py
Loading/splits	alpha_gpt/data/loader.py, alpha_gpt/config.py
Alpha language	operators/alpha_ops.py, gp_search/primitives.py, seed_injector.py
Terminal selection	gp_search/terminal_selection.py
Idea/formula debate	alpha_gpt/debate/{prompts,agents,moderator,models}.py
GP search	alpha_gpt/gp_search/engine.py
Factory	factory/{generators,verifier,run,parallel,fast_metrics,db}.py, scripts/run_factory.py
Portfolio	portfolio/construct.py, scripts/build_portfolio.py
Eval primitives	analysis/metrics.py, backtest/backtester.py, analysis/visualize.py

3 Stage 0 — Dataset construction

3.1 Sources

- **CRSP CIZ daily** (data/crsp_daily.csv, ~26 GB, ~50M rows, 25,306 PERMNOs, 2000–2025): OHLC, returns (DlyRet already includes delisting returns in CIZ), volume, shares, market cap, bid/ask, and share-class / exchange / delisting metadata.

- **Compustat** quarterly fundamentals (~ 700 line items keyed by `gvkey`, with `rdq` earnings-announcement date) and the WRDS Financial Ratios file (keyed by both `gvkey` and `permno`, with `public_date`).
- **OSAP** (Open Source Asset Pricing): ~ 200 published firm-level signals, monthly, keyed by `permno`, `yyyymm`.

3.2 Point-in-time, survivorship-free universe

The original code chose a static 500-stock universe using *full-sample* filters (look-ahead + survivorship). Replaced with a **two-pass streaming build**:

- **Pass 1** streams CRSP in chunks, drops the $\sim 23\%$ header-duplicate rows (`SecurityHdrFlg='Y'`), and reduces to a per-`(PERMNO,year)` summary (mean market cap + eligibility). Eligibility = US common stock on a major exchange: `ShareType=NS`, `SecurityType=EQTY`, `IssuerType` $\in$ `{CORP,REIT}`, `PrimaryExch` $\in$ `{N,A,Q}`, `USIncFlg=Y` ($\approx$ classic CRSP shred 10/11).
- **Annual selection**: at each calendar year take the top- N (default 1,500) eligible names by *prior-year* mean market cap — strictly trailing, so no future information enters selection.
- **Pass 2** streams CRSP again, keeps only member rows, dedups `(PERMNO,date)`, pivots each field to a `date` $\times$ `PERMNO` panel.
- **Membership mask** = in-universe-that-year *and* actually trading (`close.notna()`); every panel is masked, and `forward_returns = returns.shift(-1)` is computed from the *masked* returns (self-consistent; no out-of-universe leak).

Result: **2,752 names** over 2010–2023 ($\approx 1,500/\text{yr}$ with turnover), with **2,183 mid-sample exits and 1,146 entries** — genuinely survivorship-free. `scripts/verify_data.py` asserts this (10 checks).

3.3 Derived panels

- **Price/volume** (13 panels): `close`, `open`, `high`, `low`, `volume`, `returns`, `price`, `shrout`, `market_cap`, `bid`, `ask`, `num_trades`, `dollar_volume`.
- **Compustat** $\rightarrow$ `cs_*`: `gvkey` $\rightarrow$ `permno` from the ratios file (no CCM download); many-to-one resolved by date-overlap + in-universe + largest-cap tie-break. PIT date = `rdq` (else `datadate+90d`). Derives ~ 17 standard characteristics (gross profitability, ROA/ROE, margins, leverage, B/M, E/P, S/P, asset growth, accruals, ...), filled to daily, masked.
- **OSAP** $\rightarrow$ `osap_*`: each monthly signal becomes a panel. *Critical PIT control*: a month- M signal is stamped active on `month_end(M)+1` day, then filled to daily (a January signal only affects February onward).
- **Sector** (GICS) $\rightarrow$ `data/aux/sector.parquet` — kept *out* of `data/panels/` so it is never a numeric terminal.

3.4 Terminal discovery + IC cap

With ~ 50 – 250 candidate panels, feeding all of them to the search explodes it. `discover_terminals` lists candidates; `select_terminals` keeps a stable core plus the highest- $|\text{IC}|$ extras up to `config.max_terminals` (default 40), where IC is the fast vectorized rank-correlation on train. The full library stays on disk; each run uses the most predictive subset.

3.5 Anti-look-ahead controls (summary)

Risk	Control
Survivorship	trailing-only selection; names retained through delisting
Universe selection look-ahead	prior-year cap; no full-sample filters
Fundamentals timing	<code>rdq</code> / <code>public_date</code> (already release-lagged)
Monthly→daily fills	OSAP shifted +1 month; mask after ffill
Delisting returns	embedded in <code>CIZ DlyRet</code> ; captured as prior-day forward
Test leakage	selection on train/val only; test reserved for final backtest

4 Stage 1 — Idea generation

Three interchangeable strategies feed the same downstream pipeline.

Autonomous LLM generator (factory default). Each call samples a *theme* (momentum, value, quality, accruals, low-vol, liquidity, size, growth, ...), asks the model to invent one original cross-sectional idea in that theme and emit *k* formulas using *only* the run’s terminals, and parses strict JSON `{idea, hypothesis, expressions[]}`. Diversity from theme sampling + temperature 0.9. Token usage is recorded on every call (incl. parse failures).

Multi-agent Idea Debate (optional, higher quality). Three agents with distinct priors (Momentum, Mean-Reversion, Fundamental) run draft → cross-review (8 scored dimensions) → revision; a moderator synthesizes 2–3 hypothesis specs (~10–12 LLM calls). Used by `main.py --mode debate-only|full`.

Random generator (no API). Samples valid random expression trees — stress-tests the whole loop for \$0 and provides a noise baseline.

5 Stage 2 — Formula generation & the alpha language

The DSL. An alpha is a symbolic expression over *terminals* (data panels) and *operators*: time-series (per stock, curried windows) `ts_mean/std/delta/rank/min/max` ($w \in \{5, 10, 20, 60\}$), `ts_returns`, `ts_corr`; cross-sectional (per day) `cs_rank`, `cs_zscore`; element-wise `add`, `sub`, `mul`, `safe_div`, `log_abs`, `abs_val`, `sign`, `neg`. All operate on `date × PERMNO` frames and are NaN-safe.

Parsing. `normalize.expression` maps LLM-style `ts_delta(close,5)` to curried `ts_delta_5(close)` and aliases; the optional **Formula Debate** mirrors the idea debate at the formula level with a hard parse-gate.

6 Stage 3 — Verification

`verify_expression` (`factory/verifier.py`) is the gate that keeps the DB clean at scale: (1) parse; (2) structure (≥ 1 terminal, $\text{depth} \leq 8$); (3) safe eval on train; (4) output sanity — reject `all_nan`; require coverage $\geq 5\%$ and median breadth ≥ 20 ; require cross-sectional variation on $\geq 30\%$ of days (a constant signal has undefined IC). Returns the evaluated signal so the evaluator does not recompute it.

7 Stage 4 — Evaluation (train/val) & metrics

Splits. Train 2010–2017, validation 2018–2020, test 2021–2023.

Definitions. For a daily cross-section of alpha scores α_t and next-day returns r_{t+1} :

$$\begin{aligned} \text{IC}_t &= \text{Spearman}(\alpha_t, r_{t+1}), & \overline{\text{IC}} &= \frac{1}{T} \sum_t \text{IC}_t, \\ \text{ICIR} &= \frac{\text{mean}_t(\text{IC}_t)}{\text{std}_t(\text{IC}_t)}, & t\text{-stat}(\overline{\text{IC}}) &= \text{ICIR} \cdot \sqrt{T}. \end{aligned}$$

Turnover = mean daily $|\Delta \text{cross-sectional rank}|$; coverage = fraction of non-NaN cells. **Note:** ICIR as computed is *daily*; it annualizes as $\text{ICIR}_{\text{ann}} = \text{ICIR}_{\text{daily}} \cdot \sqrt{252}$.

Two evaluators (deliberately different speeds).

- **Fast/vectorized** (`factory/fast_metrics.py`) for *selection* during generation: IC via rank-then-Pearson over the whole panel at once (no per-date Python loop); Sharpe on a rank-weighted dollar-neutral factor return $w_t = (\text{rank}(\alpha_t) - \overline{\text{rank}}) / \sum_i |\cdot|$, $\text{ret}_t = \sum_i w_{it} r_{i,t+1}$. $\sim 50\times$ faster; good enough to *rank* alphas.
- **Exact** (`analysis/metrics.py`, `backtest/backtester.py`) for the final portfolio: Spearman per day; quintile long-short backtest.

Stored per alpha: train/val IC, train/val ICIR, val Sharpe / annual return / max drawdown, turnover, coverage, depth, generation tokens. **Test metrics are not computed here.**

8 Stage 5 — GP amplification (optional)

Seed a DEAP genetic-programming search with the LLM’s parsed expressions and evolve them (tournament selection, subtree crossover, point mutation, depth-limited; fitness = mean train IC). Each idea can spawn dozens–hundreds of verified candidates at \$0 marginal LLM cost — the cheapest path to million-scale.

9 Stage 6 — Storage

SQLite (`factory/db.py`), one row per alpha: **hash** (unique; dedup on normalized expression), **expression**, **idea**, **source**, the metrics, generation tokens, **status**. **Only expressions + scalars are stored, never signals** (1M signals would be terabytes); signals are recomputed on demand for the filtered survivors. `query()` filters by $|\text{IC}|$, $|\text{ICIR}|$, $|\text{Sharpe}|$ (a strongly negative-IC alpha is one sign-flip from a strong signal), turnover, coverage, and the **train $\wedge$ val consistency gate** (§11).

10 Stage 7 — Repeat N times (loop & parallelism)

`factory/parallel.py` has three phases: (1) **generation** — thread-pool the LLM calls (I/O-bound, near-linear speedup); (2) **evaluation** — thread-pool verify+eval. *Threads, not processes*, because the panels are GBs (processes would copy them per worker and exhaust RAM); NumPy/SciPy release the GIL during heavy ops, and the vectorized metrics keep each task short. (3) **store** — single-writer dedup + insert.

Throughput / cost (DeepSeek-V3.2 @ OpenRouter, observed). ~ 218 tokens-in + ~ 70 tokens-out per alpha (5 alphas/idea amortizes the ~ 800 -token prompt) $\Rightarrow \sim \$0.000076$ / alpha. 1,000 alphas $\approx \$0.06$, ~ 13 min. Money scales trivially ($1M \approx \$75$); **wall-clock is the real constraint** $\Rightarrow$ GP amplification and/or more eval workers.

11 Stage 8 — Portfolio construction

1. **Filter survivors.** Default/recommended: a **train^val consistency gate** ($\text{train_ic} \cdot \text{val_ic} > 0$) + a mild $|\text{val IC}|$ floor + **no top- k cap**. Empirically this beats “top- k by val IC” by a wide margin (equal-weight OOS Sharpe $0.39 \rightarrow \sim 1.2$ on the first run) because (a) breadth helps, $\text{IR} \approx \text{IC} \cdot \sqrt{\text{breadth}}$, so do not cap; and (b) the highest-val-IC alphas are often the most *overfit* — a stricter val-IC bar alone is the wrong lever; requiring the signal to hold on *both* splits removes the lucky ones.
2. **Re-evaluate** each survivor on the **test** panels $\rightarrow$ signal.
3. **Sign-flip** by $\text{sign}(\text{val_ic})$ and cross-sectionally z-score each (comparable scale).
4. **Combine** into one composite, four methods:
 - *equal* ($1/N$): NaN-aware mean of the z-scored signals.
 - *ic_weighted*: weighted mean, weights $\propto |\text{val IC}|$.
 - *decorrelated*: greedily add in $|\text{val IC}|$ order, skipping any whose signal correlation with a kept alpha exceeds `corr_threshold` (fingerprints for speed), then $1/N$ the kept set.
 - *orthogonalized*: sequentially residualize each signal against the running composite, $r_i = s_i - \beta \text{comp}$, $\beta = \langle s_i, \text{comp} \rangle / \langle \text{comp}, \text{comp} \rangle$, then average residuals (removes redundancy among similar ideas).

12 Stage 9 — Portfolio backtest & metrics

The composite is backtested *once* on test with the exact long-short quintile backtester: each day rank stocks, equal-weight long the top quintile and short the bottom, daily LS return, then annualized Sharpe $\text{SR} = \sqrt{252} \mathbb{E}[r^{LS}] / \text{std}(r^{LS})$, annual return, max drawdown, plus IC/ICIR/turnover. Baselines: the best single alpha and the value-weighted market. Outputs a comparison table (`.md` + `.json`) and an equity-curve plot.

13 End-to-end runbook

```
# Dataset (occasional)
python scripts/prepare_data.py --universe-size 1500
python scripts/ingest_fundamentals.py
python scripts/ingest_osap.py                # after downloading data/osap_signals.csv
python scripts/verify_data.py
python -m alpha_gpt.data.coverage

# Factory (repeatable)
python scripts/run_factory.py --n 200 --alphas-per-idea 5 --source llm \
```

```
--parallel --gen-workers 8 --eval-workers 10 --db data/alphas.db

# Portfolio (consistency gate + no cap are the defaults)
python scripts/build_portfolio.py --db data/alphas.db --min-train-ic 0.005
```

14 Empirical result (first unpolished 1,000-alpha run)

Generation: 200 LLM calls $\rightarrow$ 194 ideas $\rightarrow$ 970 expressions; 811 stored, 6 dup, 153 rejected by the verifier. 218k tokens, **\$0.062**, **13.2 min**. Per-alpha validation signal is weak (median $|\text{val IC}|$ 0.004); the single highest-val-IC alpha *lost 16% OOS*. Portfolio (consistency gate, 140 alphas), out-of-sample (test 2021–2023):

Method	#	IC	ICIR _d	Sharpe	Ann.Ret	MaxDD
best single	1	−0.002	−0.01	−0.69	−10.6%	−36%
equal (1/N)	140	0.025	0.16	1.43	26.3%	−14%
ic_weighted	140	0.023	0.15	1.37	24.7%	−16%
decorrelated	99	0.026	0.17	1.40	26.0%	−14%
orthogonalized	140	0.029	0.16	1.44	33.9%	−22%

Like-for-like, the equal-weight portfolio has daily ICIR 0.106 ($\Rightarrow$ annualized ≈ 1.7), IC t -stat ≈ 4.3 , Sharpe t -stat ≈ 2.5 , positive in all three years.

Takeaways. (1) the single “best” alpha overfits val $\rightarrow$ test; (2) combining weak, decorrelated signals yields a real factor with a *shallower* drawdown; (3) selection $>$ combiner, and cross-split consistency $>$ raw magnitude.

15 Reading the results — is Sharpe 0.88–1.4 “good”?

Reference points (annualized, **gross** unless noted): S&P 500 long-only ~ 0.4 – 0.6 ; a typical single published anomaly ~ 0.3 – 0.6 ; a well-built multi-factor tangency portfolio ~ 1.0 – 1.5 ; top quant funds ~ 1 – 2 *net of costs*. So an OOS gross Sharpe in this range from a completely unpolished automated pipeline is *encouraging* — in the “real signal, not noise” range, beating the market with a smaller drawdown — but it is **not** a validated edge, because:

- **Gross, not net.** Turnover $\approx 0.1/\text{day}$ is high; realistic costs could pull net Sharpe toward ~ 0.5 – 0.9 .
- **Test-peeking.** Choosing the selection rule / construction method / thresholds by their test score inflates the number; the consistency gate is principled (validation-side), but specific values tuned on test are optimistic.
- **One test window, one run** $\Rightarrow$ high variance.
- No borrow/capacity/neutralization modeled.

The robust finding is the *shape* (portfolio $\gg$ best single; consistency gate $\gg$ magnitude ranking; beats benchmark across regimes), which is exactly what sound alpha-combination looks like.

16 Limitations & roadmap

- **Net-of-cost backtest** ($\text{turnover} \times \text{spread}$) — the most important missing piece for credibility.
- **Validation rigor** — walk-forward / multiple windows / multiple seeds; deflated Sharpe & multiple-testing control given many candidates.
- **Neutralization** — `analysis/neutralize.py` exists but is not yet wired into eval/backtest.
- **Incremental selection** — score each new alpha by marginal contribution to the existing portfolio.
- **Scale** — GP amplification + more eval workers toward 1M; the cost is wall-clock, not dollars.
- **Richer data axes** — IBES (analyst), OptionMetrics (IV), 13F, short interest, each a new orthogonal terminal family.